

STACKS BITCOIN

RiskOS

Complete Technical Implementation Blueprint

Unified Position API / Stacks-native Risk Engine / Protective Execution /
Distribution SDK

Architecture baseline: 3 September 2026

Prepared for implementation, grant scoping, protocol partnerships and security review

Document purpose

This blueprint defines everything required to turn RiskOS from a product concept into a fully functional, non-custodial, integration-ready Bitcoin finance risk platform on Stacks. It covers product boundaries, onchain and offchain architecture, protocol adapters, data contracts, risk formulas, transaction preparation, SDK distribution, smart-contract decisions, security, reliability, testing, legal/compliance workstreams, operations, staffing, cost, delivery phases and acceptance criteria.

Core architecture decision. The production MVP should not deploy a custody contract and should never hold user keys. RiskOS reads public state, computes explainable risk, builds unsigned or wallet-requested transactions, simulates outcomes, and requires the user or institutional policy engine to sign. New Clarity contracts are optional only when bounded automation, delegated permissions or onchain attestations become necessary.

This is an implementation blueprint, not legal, investment or security advice. Protocol contracts, parameters and APIs can change. Every mainnet address must be taken from a versioned, signed integration registry and independently checked against the protocol's official repository, explorer and partner confirmation before release.

How to read this report

- Sections 1-5 define scope, architecture and the four product modules.
- Sections 6-13 define protocol integrations, data, risk methodology, contracts and execution.
- Sections 14-22 define APIs, SDKs, security, operations, testing and compliance.
- Sections 23-28 define delivery, team, budget, partner dependencies and launch gates.
- Appendices provide endpoint, event, database, rule, runbook and source checklists.

Evidence status labels

Label	Meaning	Engineering treatment
Verified public interface	Documented contract, API, SDK or repository is publicly available.	Implement and pin a tested version.
Partner confirmation required	The capability exists but public documentation is incomplete, recently changed or lacks a stable machine interface.	Obtain schema, addresses, rate limits, support contact and change-notice commitment.
Derived metric	RiskOS calculates the value from raw state and documented parameters.	Expose formula, inputs, timestamp, confidence and version.
Roadmap dependency	Underlying protocol capability is announced or evolving.	Do not make it a launch blocker; feature-flag it.

Contents

Document purpose	2
How to read this report	2
Evidence status labels	2
Contents	3
1. Product definition and non-negotiable boundaries	7
1.1 Product promise	7
1.2 Launch personas	7
1.3 Hard boundaries	7
1.4 Definition of 'fully fledged'	7
2. System architecture	8
2.1 Logical services	8
2.2 Recommended implementation stack	9
2.3 Deployment environments	9
3. Module 1 - Unified Position API	10
3.1 Required outputs	10
3.2 Position envelope	10
3.3 Data correctness rules	10
3.4 Public vs authenticated access	11
4. Protocol adapter implementation	12
4.1 Adapter interface	12
4.2 Integration matrix	12
4.3 sBTC adapter state machine	13
4.4 Zest v2 specifics	13
4.5 Granite specifics	13
4.6 Bitflow specifics	13
5. Module 2 - Stacks-native risk engine	15
5.1 Risk-result contract	15
5.2 Lending equations	15
5.3 Liquidation liquidity	15
5.4 LP risk methodology	15
5.5 Bridge and signer risk	16
5.6 Oracle risk	16
5.7 Staking and bond risk	16
5.8 Portfolio concentration graph	16
5.9 Risk score	17

6. Module 3 - Protective execution layer	18
6.1 Action workflow	18
6.2 Supported MVP actions	18
6.3 Transaction intent schema	18
6.4 Guardrails	18
6.5 Wallet and custody integrations	19
7. Smart-contract decision: what RiskOS must deploy	20
7.1 MVP answer - no RiskOS contract required	20
7.2 Optional Phase 3 contracts	20
7.3 Why automation still needs offchain infrastructure	20
7.4 Contract engineering requirements	20
8. Module 4 - Distribution SDK and widget	21
8.1 Deliverables	21
8.2 Integration experiences	21
8.3 Widget security	21
8.4 Commercial API controls	21
9. API surface and behavior	22
9.1 Core endpoints	22
9.2 Error model	22
9.3 API rules	22
10. Data architecture	23
10.1 Core tables	23
10.2 Event identity and idempotency	23
10.3 Data lineage	23
10.4 Backfills and replays	23
11. Pricing, APY and incentive calculations	24
11.1 Price hierarchy	24
11.2 Effective net APY	24
11.3 90-day incentive implementation	24
12. Authentication, teams and institutional workflow	25
12.1 Retail identity	25
12.2 Institutional tenancy	25
12.3 Approval workflow	25
13. Alerts and notification engine	26
13.1 Initial rule catalog	26
13.2 Delivery	26
14. Security architecture and threat model	27

14.1 Crown jewels	27
14.2 Threat matrix	27
14.3 Secure development lifecycle	27
14.4 Secrets and infrastructure	27
15. Reliability, SLOs and operations	29
15.1 Production service objectives	29
15.2 Dependency health	29
15.3 Incident levels	29
15.4 Required runbooks	29
16. Testing and verification strategy	30
16.1 Test pyramid	30
16.2 Independent validation	30
17. Privacy, compliance and legal workstream	31
17.1 Product posture	31
17.2 Required controls	31
17.3 USDCx integration caveat	31
18. Admin, governance and change management	32
18.1 Control-plane actions	32
18.2 Model governance	32
19. CI/CD and release engineering	33
19.1 Repository quality gates	33
20. Delivery plan	34
20.1 Grantable MVP - 16 to 20 weeks	34
20.2 Full product - following 6 to 12 months	34
20.3 Scope discipline	34
21. Team and ownership	35
21.1 Operating responsibilities	35
22. Infrastructure and operating cost	36
22.1 Non-infrastructure budget items	36
23. Partner dependency checklist	37
24. Launch acceptance criteria	38
24.1 Functional	38
24.2 Safety and reliability	38
24.3 Market validation	38
25. Principal risks to the RiskOS business	39

26. Open decisions before engineering kickoff	40
27. Recommended repository and service map	41
27.1 Domain rules	41
28. Final implementation recommendation	42
Appendix A - Complete build checklist	43
A.1 Product and partnerships	43
A.2 Engineering	43
A.3 Production readiness	43
Appendix B - Source and contract verification rules	44
B.1 Currently verified anchors	44
Appendix C - Reference links	45
Appendix D - Definition of done by module	47

1. Product definition and non-negotiable boundaries

1.1 Product promise

RiskOS gives a Bitcoin or Stacks address a continuously updated, provenance-aware view of positions, risks, rewards and exit options across sBTC, lending, liquidity and Bitcoin Staking. It then prepares the safest available user-approved action. The platform is both a first-party dashboard and infrastructure other wallets, protocols and treasury systems can embed.

The ecosystem direction supports exactly this layer: Stacks' roadmap prioritizes Bitcoin-native finance, Bitcoin-grade safety, deep liquidity, safe exit paths, self-custodial lending, sBTC bridge optimization and agent infrastructure.[\[S1\]](#)

1.2 Launch personas

Persona	Primary job	Required output
Retail BTCfi user	Know whether positions are safe and what to do next.	Plain-language risk, alert, signed protection flow.
Active LP / borrower	Optimize incentive eligibility without accidental liquidation or idle liquidity.	Time-weighted LTV, active-range status, exit slippage, scenario tool.
Treasury / fund	Aggregate exposure, maturities, approvals and audit evidence.	Multi-wallet views, policy alerts, CSV/API, immutable action log.
Wallet / protocol	Add cross-protocol intelligence without rebuilding indexers and risk models.	API, SDK, widget, webhooks, partner SLA.
Risk / operations team	Detect data failures, incidents and systemic concentration.	Market health console, provenance, incident controls.

1.3 Hard boundaries

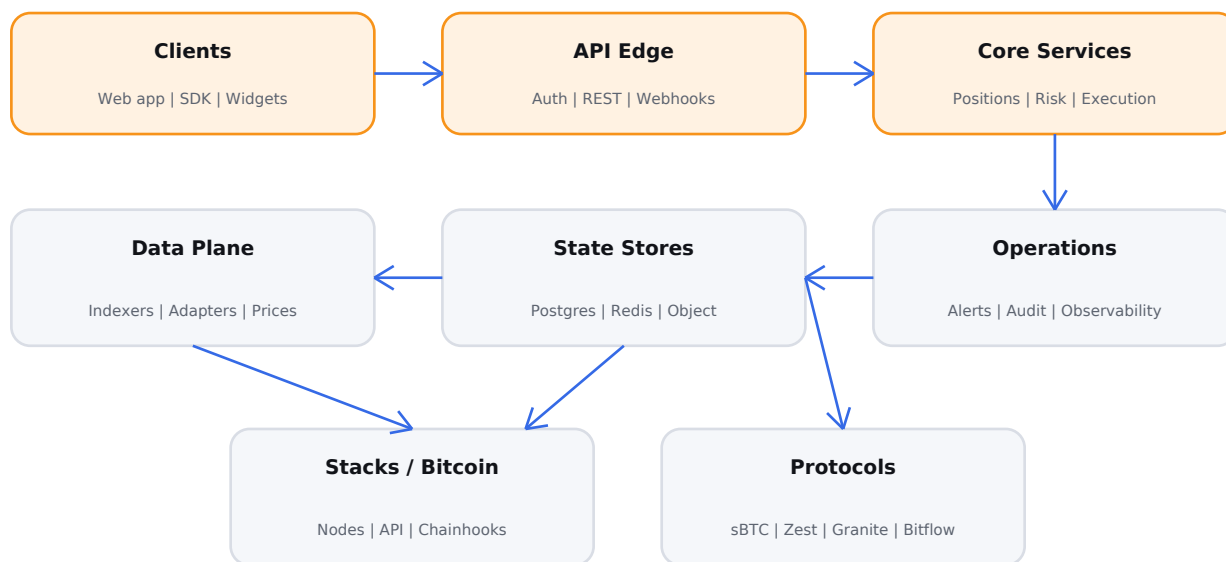
- No private keys, seed phrases or custody. Browser wallets, mobile wallets or institutional custody systems sign.
- No hidden execution. MVP actions require an explicit human signature and show contract, function, arguments, fee, minimum received, slippage and post-action risk.
- No invented precision. Every metric includes source block, observed time, computation version and confidence/degradation state.
- No universal health formula. Each protocol adapter mirrors the exact deployed contract version and risk parameters.
- No hardcoded mainnet addresses in application logic. Addresses and ABI/Clarity interface hashes live in a signed, versioned registry.
- No transaction construction when required state is stale, disputed or unsupported. Fail closed and present a safe manual path.

1.4 Definition of 'fully fledged'

A production-grade release is not merely a dashboard. It must have reliable chain ingestion, versioned adapters, reconciled snapshots, a deterministic risk model, simulations, secure transaction construction, notifications, public and authenticated APIs, an SDK/widget, partner onboarding, an admin control plane, monitoring, disaster recovery, a security program, documented legal positioning and measurable service objectives.

2. System architecture

RiskOS uses an event-driven read model. Chainhooks and node/API polling ingest canonical Stacks and Bitcoin activity; protocol adapters decode it into normalized positions; pricing and parameter services enrich state; the snapshot ledger preserves time; the risk engine emits findings; the alert service notifies users; and the execution service builds but does not sign protective transactions. Chainhooks provides filtered, reorg-aware, historical event delivery; it should be combined with independent reconciliation rather than treated as the sole source of truth.[\[S9\]](#) [\[S10\]](#)



2.1 Logical services

Service	Responsibilities	State / dependencies
API gateway	TLS, routing, request IDs, quotas, WAF, API version negotiation.	Edge/CDN, API key store.
Identity & tenant	Wallet challenge sessions, OAuth/OIDC for teams, RBAC, API keys, service accounts.	Postgres, KMS/HSM.
Chain ingestion	Stacks blocks, microblocks/tenures, mempool, contract events, Bitcoin txs, reorg rollback.	Stacks API/node, Chainhooks, Bitcoin node/provider.
Protocol adapters	Decode protocol-specific state/events and build typed normalized records.	Versioned registry, Clarity read-only calls.
Position service	Aggregate balances, debt, collateral, LP, staking, bridge and rewards.	Postgres/Timescale, Redis.
Pricing service	Oracle reads, market references, TWAPs, staleness, disagreement and confidence.	DIA, protocol oracles, Bitflow quotes, market feeds.
Risk engine	Health, distance, scenario, concentration, dependency and eligibility calculations.	Snapshots, parameters, model registry.
Simulation & quote	Post-action state, route quote, fees, slippage, min-out and expiry.	Bitflow SDK/API, read-only calls.
Transaction builder	Validated transaction intent and wallet payloads; never signs.	Stacks.js/Connect, contract registry.
Alert & workflow	Rules, deduplication, severity, cooldown, email/push/webhook delivery.	Queue, notification providers.
Reporting & exports	CSV/PDF/JSON reports, accounting lots, audit history, maturity ladders.	Object store, job queue.
Admin/control plane	Protocol pause, feature flags, adapter versions, source health, incident banners.	Privileged RBAC, approval log.

2.2 Recommended implementation stack

Layer	Recommended baseline	Why
Web	Next.js + React + TypeScript; server rendering for public market pages.	Fast product delivery and shared types with SDK.
API	TypeScript on Fastify or NestJS; OpenAPI-first.	Typed adapters, mature validation, generated clients.
Workers	TypeScript workers initially; Rust only for high-throughput decoding/simulation hotspots.	Avoid premature split while preserving an optimization path.
Workflow	Temporal for durable multi-step jobs or BullMQ/SQS for a simpler MVP.	Retries, timeouts, idempotency and visibility.
Primary DB	PostgreSQL 16+; TimescaleDB extension for snapshots/time series.	Transactional truth plus time-weighted analytics.
Cache / locks	Redis-compatible managed service.	Low-latency reads, rate limits and distributed locks.
Analytics later	ClickHouse when event volume or portfolio history exceeds Postgres economics.	Columnar historical analytics.
Object storage	S3-compatible encrypted buckets.	Reports, raw payload quarantine, replay fixtures.
Infrastructure	AWS ECS/Fargate + RDS Multi-AZ + ElastiCache; Terraform/OpenTofu.	Production reliability without Kubernetes overhead.
Observability	OpenTelemetry + managed metrics/logs/traces + on-call alerting.	Vendor-neutral telemetry. [S32]

2.3 Deployment environments

- Local: Clarinet simnet plus mocked protocol adapters and deterministic replay fixtures.[\[S13\]](#)
- Integration: Stacks testnet plus real provider sandboxes; destructive tests allowed on isolated tenants.
- Staging: production-equivalent infrastructure using shadow mainnet reads; transaction broadcasting disabled by policy.
- Production: mainnet reads and user-signed transaction broadcasting; multi-AZ, backups, WAF and pager rotation.
- Research: isolated notebooks/data warehouse; never shares production secrets or write credentials.

3. Module 1 - Unified Position API

3.1 Required outputs

The API must return raw positions, normalized positions, valuations, provenance and confidence. A user must be able to distinguish spendable assets from collateral, debt, LP inventory, locked or pooled staking, pending bridge operations and claimable rewards. Aggregation is performed by address and optionally across a tenant-controlled wallet group.

Position family	Minimum normalized fields	User outcome
Wallet assets	asset, contract, amount, decimals, USD/BTC value, spendability.	Know what is liquid versus committed.
Lending	protocol/version, collateral, debt, accrued interest, LTV, liquidation threshold, health factor.	Know safety buffer and cost of debt.
Concentrated LP	pool, token amounts, range/ticks, active flag, fees, rewards, position value, exit quote.	Know whether capital earns and what can actually be exited.
sBTC bridge	direction, Bitcoin txid/outpoint, Stacks txid, amount, fee, confirmations, signer/Emily status, timestamps.	Understand where funds are and expected next step.
Bitcoin bond	BTC/STX commitments, lock start/end, cycle, reward schedule, realized/expected reward, early-exit effect.	Plan liquidity and return.
Pooled staking / stBTC	operator, shares, exchange rate, underlying, fee, capacity, withdrawal terms, maturity and trust model.	Compare yield after fees and operator risk.
Rewards/incentives	source, accrued/claimable/paid, eligibility, eligibility evidence, net APY.	Avoid losing incentives and separate durable yield from subsidies.

3.2 Position envelope

```
{
  "asOf": {"stacksBlock": 123456, "bitcoinBlock": 966350,
    "observedAt": "2026-09-03T15:00:00Z"},
  "address": "SP...",
  "positions": [{
    "id": "zest-v2:lending:SP...:market-0",
    "type": "lending",
    "protocol": {"id": "zest", "version": "v2", "contract": "SP...v0-8-market"},
    "assets": [{"role": "collateral", "asset": "sBTC", "amount": "0.60000000"}],
    "debt": [{"asset": "USDCx", "principal": "25000", "accrued": "41.52"}],
    "metrics": {"ltv": "0.41", "healthFactor": "1.24"},
    "provenance": [{"source": "contract-read", "block": 123456}],
    "confidence": {"state": "verified", "score": 0.99}
  ]},
  "warnings": []
}
```

3.3 Data correctness rules

- Amounts are integers at ingestion. Decimal conversion occurs only at API boundaries using registered token decimals.
- Every position is reproducible from raw event IDs plus point-in-time contract reads.
- Confirmed, unconfirmed and projected values are separate fields; never silently merge them.
- Reorg rollback invalidates affected snapshots and recomputes all dependent risk results.
- Contract upgrades create a new adapter version; historical records retain the prior interpretation.
- A daily full reconciliation compares indexed state with direct read-only calls and token balances.
- Unknown tokens and unrecognized contract versions are displayed as unsupported exposure, never ignored.

3.4 Public vs authenticated access

Mode	Allowed	Restricted
Public address lookup	Public onchain positions and aggregate risk with strict quotas.	Saved alerts, private labels, grouped treasuries, exports, webhooks.
Wallet session	Watchlists, thresholds, preferences, transaction requests.	Other tenants' labels, team policies and audit logs.
B2B API key	Scoped addresses, market feeds, risk and webhooks.	Human interactive signing; tenant-admin functions unless separately scoped.
Institutional tenant	Wallet groups, roles, approvals, policy events, audit exports.	Key custody unless provided by the customer's custody platform.

4. Protocol adapter implementation

4.1 Adapter interface

```
interface ProtocolAdapter {
  manifest(): AdapterManifest;
  discover(address: Principal, at: BlockRef): Promise<PositionRef[]>;
  hydrate(refs: PositionRef[], at: BlockRef): Promise<NormalizedPosition[]>;
  parameters(at: BlockRef): Promise<RiskParameters>;
  rewards(address: Principal, window: TimeWindow): Promise<RewardRecord[]>;
  buildIntent(action: ProtectiveAction, state: Snapshot): Promise<TxIntent>;
  simulate(intent: TxIntent, state: Snapshot): Promise<SimulationResult>;
  reconcile(address: Principal, at: BlockRef): Promise<ReconciliationResult>;
}
```

Each manifest pins network, deployer, exact contract principal(s), interface hash, supported assets, start block, event decoders, read-only functions, transaction functions, feature flags and deprecation date. A four-eyes approval publishes registry changes.

4.2 Integration matrix

Adapter	Read path	Write/action path	Launch status
Stacks core	Stacks API + own node; SIP-010 balances; tx/events; mempool.	Stacks Connect / wallet RPC broadcast. [S10] [S11] [S12]	Mandatory, public interface.
sBTC	Token state + registry contracts + Emily/bridge API + Bitcoin confirmation source.	Deposit/withdrawal request payload; user signs BTC or Stacks step. [S5] [S6] [S7] [S8]	Mandatory; integration tests against live state.
Zest v2	Events + v0 market/vault/assets/egroup read-only calls; versioned deployment manifest.	repay, collateral-add/remove and related v0-8-market calls. [S14] [S15] [S16]	Mandatory. Very recent contracts require partner verification.
Granite	Open-source contracts, events and math SDK; direct reads.	repay/add collateral/withdraw flows per verified current contracts. [S17] [S18]	Mandatory after verified address/interface workshop.
Bitflow	SDK/API for routes and quotes; pool contracts/events for LP state.	swap and LP management payloads through SDK/verified contracts. [S19] [S20] [S21]	Mandatory for MVP; LP schema and rate limits require partner agreement.
Stacking DAO stBTC	SIP-010 balance, exchange-rate/backing contracts, deposit/redemption state.	deposit/redeem/claim if publicly supported. [S22] [S23] [S37]	Phase 2; code is evolving, confirm production manifest.
PoX-5 / Bitcoin Bonds	PoX/bond contracts, Bitcoin timelock tx and reward outputs; protocol/pool data.	Primarily reporting; early exit/bond actions only from official workflow. [S2] [S3]	Phase 2; bootstrap is managed and product-specific.
DIA	get-value(key), value and timestamp from mainnet oracle.	Read only. [S24]	Mandatory secondary/reference source.
Pyth Lazer/Core	Read protocol-verified Lazer inputs and timestamps; do not assume Core availability.	No independent onchain update in MVP. [S15] [S25]	Lazer via Zest; Core marked unsupported by Pyth as of 26 Aug 2026.

4.3 sBTC adapter state machine

RiskOS state	Evidence	Timeout / alert
deposit_detected	Bitcoin tx/outpoint observed.	Alert if not confirmed within user-selected BTC confirmation window.
deposit_confirming	Bitcoin confirmations increasing.	Track replacement/double-spend and reorg.
deposit_accepted	Emily/signers recognize request.	Alert on mismatch or missing request after confirmations.
deposit_swept	Signer sweep spends deposit output.	Track sweep confirmation.
mint_pending	Sweep confirmed; Stacks completion not yet final.	Escalate against rolling p95 bridge time.
completed	sBTC mint/transfer confirmed and reconciled.	Close alert only after balance match.
withdrawal_requested	Stacks contract request confirmed.	Track amount, recipient and fee.
withdrawal_signing	Signer action pending.	Show signer/network degradation rather than promise an ETA.
withdrawal_broadcast	Bitcoin payout tx observed.	Track confirmations and RBF/replacement.
failed_or_refunded	Terminal failure or refund path verified.	Provide official recovery link and support evidence bundle.

Emily is a liaison between users and sBTC signers; RiskOS must not treat an Emily response alone as final settlement. Final state is reconciled against Bitcoin and Stacks onchain evidence.[\[S7\]](#) [\[S8\]](#)

4.4 Zest v2 specifics

- Use the current v0-8-market as the primary orchestrator and v0 market-vault as the position ledger; read asset and egroup registries for per-combination parameters.[\[S15\]](#) [\[S16\]](#)
- Support traditional and rehypothecatable collateral separately. zToken balances can be collateral and continue accruing supply yield; do not double-count underlying value.[\[S14\]](#)
- Mirror the protocol's egroup selection before calculating capacity, LTV or liquidation state. Adding a new collateral type can change the applicable egroup.
- Record optional in-band Pyth Lazer price payload requirements on transaction intents. A stale quote cannot be converted into a valid repay/borrow action without the appropriate feed handling.
- Pin the 31 August 2026 market/data deployments and 2 September 2026 stBTC strategy contracts only after partner confirmation; their freshness increases integration risk.[\[S15\]](#)
- For strategy vaults, expose leverage loop, borrowed asset, underlying LST, protocol dependencies and unwind liquidity as separate exposures.

4.5 Granite specifics

- Obtain and validate current mainnet/testnet contract manifest, supported markets, oracle source, decimals, LTV/soft-liquidation math and event schemas from Granite before coding the production adapter.
- Use Granite's math SDK only as a convenience; validate results against deployed Clarity read-only functions and golden test vectors.
- Model isolated pools independently. Never net collateral or debt across pools unless the contracts do so.
- Represent soft-liquidation state and price impact distinctly from binary liquidation; show the portion exposed and expected recoverable collateral.
- Add a protocol pause switch if contract version or oracle behavior is not recognized. Granite is open-source and operates an active bounty, but public source is not a substitute for a signed integration manifest.[\[S17\]](#) [\[S18\]](#)

4.6 Bitflow specifics

- Use the SDK/public quote service for swaps and verified pool contracts/events for authoritative LP ownership. Never infer ownership solely from cached API responses.[\[S19\]](#) [\[S20\]](#)
- Index pool creation, liquidity changes, position ownership/transfers, fee collection, reward staking and range parameters from the relevant contract generation.

- At quote time capture route, pools, price impact, fees, expected output, minimum output, quote expiry, block and SDK version.
- For exit slippage, quote the actual underlying amounts produced by removing liquidity and, separately, the optional conversion of both legs into the user's chosen numeraire.
- Build a partner contract covering API authentication, quotas, SLA, schemas, deprecation notice, route reproducibility, historical access and incident contacts. Public docs advertise SDK/API resources but do not by themselves guarantee stable LP analytics interfaces.[\[S21\]](#)

5. Module 2 - Stacks-native risk engine

5.1 Risk-result contract

```
{
  "riskId": "rk_...",
  "positionId": "...",
  "severity": "high",
  "category": "liquidation",
  "score": 82,
  "title": "sBTC collateral is 11.4% above liquidation",
  "evidence": [{ "metric": "healthFactor", "value": "1.09", "sourceBlock": 123456 }],
  "scenarios": [{ "name": "BTC -10%", "healthFactor": "0.98" }],
  "recommendedActions": [{ "type": "repay", "amount": "2800 USDCx" }],
  "model": { "id": "zest-v2-liquidation", "version": "1.3.0" },
  "confidence": { "state": "verified", "score": 0.98 },
  "expiresAt": "2026-09-03T15:01:00Z"
}
```

5.2 Lending equations

For protocol p , define collateral value $C = \sum(\text{amount}_i \times \text{price}_i)$, debt $D = \sum(\text{debt}_j \times \text{price}_j)$, protocol maximum LTV L , liquidation threshold T , and risk-adjusted collateral $A = \sum(\text{collateralValue}_i \times \text{liquidationThreshold}_i)$. Use the protocol's integer rounding order and egroun/market rules exactly.

Metric	Definition	Engineering note
LTV	D / C	Undefined when $C=0$; return null, not zero.
Health factor	A / D	Safe above 1 under the applicable protocol rules; verify exact protocol semantics.
Liquidation buffer	$(A - D) / A$	Explain as percent reduction in adjusted collateral before threshold.
Single-asset liquidation price	Solve $D = \text{otherAdjustedCollateral} + \text{quantity} \times \text{price} \times T$	Only valid when debt/other prices are held constant.
Repay to target HF h	$\max(0, D - A/h)$, adjusted for fees and interest	Recompute using protocol rounding and transaction-time interest.
Add collateral to target HF h	$\max(0, (hD - A)/(\text{price} \times T))$	Include price uncertainty and transfer fee if relevant.
Time-weighted LTV	$\text{integral}(\text{LTV}(t)dt) / \text{integral}(dt)$	Event-sourced intervals; daily snapshots alone may be insufficient for program rules.

5.3 Liquidation liquidity

RiskOS must distinguish protocol solvency from market executability. For each collateral/debt pair, compute available executable liquidity at 0.5%, 1%, 2% and 5% impact using Bitflow routes and directly observed pool depth. Estimate liquidation proceeds net of pool fees, slippage, liquidation bonus and Stacks transaction fees. Publish the quote time and never label a stale quote as available liquidity.

5.4 LP risk methodology

Metric	Method	Caveat
In-range state	Compare current pool price/tick with position lower and upper bounds.	Use pool's exact tick/price convention.
Range probability	Calibrated historical simulation or GBM baseline using realized volatility, horizon and current distance to bounds.	A forecast, not a guarantee; expose model and backtest error.
Capital efficiency	Active liquidity value / total position value, plus fees per active-dollar-hour.	Separate capital deployment from profitability.
Impermanent loss	Current withdrawable LP value minus value of original token quantities held, with fees/rewards shown separately.	Requires deposit-lot history; address import may lack original cost basis.
Exit slippage	Expected realized value from remove-liquidity plus optional swaps compared with mid/reference value.	Route size and quote age are mandatory.
Fee sustainability	Annualized trailing fees net of incentives and gas, using multiple windows.	Do not extrapolate a short incentive burst as durable APY.

5.5 Bridge and signer risk

- Backlog: counts and value of accepted but incomplete deposits/withdrawals by age bucket.
- Latency: p50/p90/p99 completion time by direction and rolling window.
- Signer availability proxy: observed signer-driven operations and progress rate; label as inferred unless an official health endpoint is available.
- Reserve/issuance consistency: compare circulating sBTC and publicly verifiable backing/state using official mechanisms; do not invent a proof-of-reserve method.
- Concentration: value exposed to the sBTC signer model across wallet, collateral, LP and pooled products, avoiding double-counting nested assets.
- Alert modes: delayed, stalled, source disagreement, reorg, unexpected fee, unsupported recipient, terminal failure.

5.6 Oracle risk

DIA's Stacks oracle exposes value and timestamp, with a documented mainnet address and update configuration; RiskOS reads both and calculates age. Zest v2 documents Pyth Lazer as primary and DIA for selected assets. Pyth's Core guide says Core support for Stacks ended on 26 August 2026, so the product must distinguish Core from Lazer and never describe them as one source.[\[S15\]](#) [\[S24\]](#) [\[S25\]](#)

Check	Trigger example	Response
Freshness	age exceeds protocol threshold or RiskOS stricter threshold.	Degrade confidence; block protective build if protocol would reject price.
Cross-source disagreement	absolute difference exceeds 1% or asset-specific band.	High warning; calculate both outcomes.
Market divergence	oracle differs materially from executable Bitflow/venue median.	Show liquidation may use oracle while exits use market price.
Timestamp regression	new observation timestamp is older than prior accepted value.	Quarantine source and page operator.
Single-source dependency	only one healthy source remains.	Amber system state; widen safety recommendations.
Stablecoin deviation	USDCx market price or redemption evidence diverges from \$1.	Scenario debt/collateral asymmetrically; show xReserve dependency.

5.7 Staking and bond risk

- Maturity ladder: BTC and STX committed by unlock block/date, expected distributions and liquidity gaps.
- Early-exit cost: forfeited future BTC yield, transaction fees and any operator-specific penalty; STX remains locked for the term in the documented native design.[\[S2\]](#)
- Yield realization: expected versus received BTC per weekly distribution; target APY is not guaranteed.[\[S2\]](#)
- STX/BTC reflexivity scenarios: model miner BTC commitments, STX price, coverage band and reserve behavior; clearly label protocol parameter assumptions.[\[S2\]](#)
- Pool/operator risk: custody model, operator, fee, capacity, redemption queue and contract version. Pooled staking introduces both sBTC signer and operator trust.[\[S3\]](#)
- Nested exposure: stBTC used in Zest or LP positions still carries Stacking DAO, sBTC, staking and downstream protocol dependencies.

5.8 Portfolio concentration graph

Represent exposures as a directed graph rather than a flat list. A position can depend on an asset, protocol, smart contract, oracle, pool operator, signer/custody model and stablecoin issuer. Compute direct exposure, look-through exposure and stressed loss contribution. Prevent double counting by assigning each underlying economic unit a lineage ID.

Dimension	Examples	Required aggregation
Asset	BTC, sBTC, stBTC, STX, USDCx.	Gross, net and look-through amounts.
Protocol	Zest, Granite, Bitflow, Stacking DAO.	Value at risk and exit dependency.

Dimension	Examples	Required aggregation
Contract/version	v0-8-market, vault, pool generation.	Exact value exposed per code surface.
Oracle	Pyth Lazer, DIA, protocol-specific feed.	Positions liquidatable using each source.
Custody/trust	self-custodial BTC lock, sBTC signer set, pool operator.	Value under each assumption.
Issuer/bridge	Circle/xReserve for USDCx.	Stablecoin and settlement dependency.

5.9 Risk score

The 0-100 score is a presentation layer, never the source of truth. Store component scores for liquidation, liquidity, oracle, bridge, contract, operator, maturity and concentration. Severity overrides apply to imminent liquidation, bridge failure, unsupported contract upgrades and oracle disputes. Publish weight/version changes and preserve old results for auditability.

6. Module 3 - Protective execution layer

6.1 Action workflow

Stage	System responsibility	Failure rule
Detect	Risk rule emits evidence and allowed remediations.	No recommendation without fresh, reconciled state.
Plan	Generate one or more action plans with amounts and post-action targets.	Show alternatives and assumptions.
Quote	Obtain executable route/fee/min-out and expiry where needed.	Expired quote cannot be signed.
Simulate	Run read-only/contract-aware state transition and recompute portfolio risk.	Block if simulation fails or worsens another critical risk.
Explain	Display contract, method, assets, approvals, fees, slippage and before/after metrics.	Unknown contract/function is a hard stop.
Request signature	Send structured wallet request through supported connector.	No key material reaches RiskOS.
Broadcast	Wallet or custody platform broadcasts; capture txid.	Idempotency key prevents duplicate workflow.
Track	Monitor mempool, confirmation, failure and replacement.	Alert user if state changed before confirmation.
Reconcile	Verify resulting balances/position and close or reopen risk.	Never infer success from a submitted tx alone.

6.2 Supported MVP actions

Action	Inputs	Post-condition
Repay Zest/Granite debt	market, debt asset, amount, source balance, price payload if required.	Debt reduced and target HF/LTV achieved within rounding tolerance.
Add sBTC collateral	market, amount, token approval/transfer path, parameter set.	Collateral credited and health improved.
Swap through Bitflow	from/to, amount, route, min-out, expiry, max slippage.	Received amount $\geq$ min-out and route contracts match registry.
Remove LP	position, percentage, min token outputs, fee claim behavior.	Liquidity removed and token amounts reconciled.
Convert rewards	claim source, destination policy, swap route.	Rewards claimed and converted within policy.
Withdraw sBTC	amount, Bitcoin address/network, fee, bridge request.	Request confirmed then Bitcoin payout tracked to finality.

6.3 Transaction intent schema

```
{
  "intentId": "int...", "network": "mainnet", "expiresAt": "...",
  "reason": {"riskId": "rk...", "targetHealthFactor": "1.35"},
  "calls": [{
    "contract": "SP...v0-8-market", "function": "repay",
    "args": ["..."], "postConditions": ["..."], "fee": "..."
  }],
  "guardrails": {"maxSlippageBps": 50, "minReceived": "...", "maxFee": "..."},
  "simulation": {"status": "passed", "stateBlock": 123456, "postRiskScore": 28},
  "registryVersion": "2026-09-03.4", "adapterVersion": "zest-v2@1.3.0"
}
```

6.4 Guardrails

- Exact allowlist of contract principal + function + interface hash; deny arbitrary contract calls.
- Stacks post-conditions whenever the action supports them; constrain transferred assets and amounts.
- Quote expiry and maximum state-block drift before signature; require re-plan if exceeded.
- Per-action and daily value limits for future delegated execution.
- Minimum post-action health, minimum received, maximum slippage and maximum fee.
- Simulation must account for accrued interest, oracle update requirements and existing pending transactions.
- Human-readable review is generated from the same typed intent used to construct the transaction.

- Kill switches by protocol, contract, asset, action and tenant.

6.5 Wallet and custody integrations

Retail web flows should use the current Stacks Connect request interface, which provides wallet compatibility. Xverse additionally exposes a developer SDK for Bitcoin wallet actions. Institutional distribution should hand structured policy events and transaction requests to existing custody workflows such as Fireblocks instead of attempting to become a custodian.[\[S11\]](#) [\[S12\]](#) [\[S28\]](#) [\[S29\]](#)

7. Smart-contract decision: what RiskOS must deploy

7.1 MVP answer - no RiskOS contract required

RiskOS can deliver all four modules without deploying a new smart contract. Position discovery and risk are read-only. Protective actions call the user's existing protocol contracts. The wallet signs. The SDK/widget is offchain distribution infrastructure.

Capability	New contract?	Reason
Read balances/positions	No	Read existing tokens and protocol state.
Compute risk and incentives	No	Offchain deterministic analytics with onchain evidence.
Prepare repay/add collateral/swap/remove LP	No	Build calls to existing verified contracts.
User-signed protection	No	Wallet/custodian authorizes the existing protocol call.
Alerts/webhooks/reports	No	Offchain services.
Scheduled autonomous execution	Possibly	Needs bounded authorization plus an offchain keeper; wallet capabilities may also satisfy it.
Onchain risk attestations	Optional	Only needed if another contract consumes RiskOS output.
Partner incentive distribution	Optional	Only if RiskOS becomes a distributor rather than an eligibility/data provider.

7.2 Optional Phase 3 contracts

Contract	Purpose	Essential controls
riskos-policy-v1	User-defined delegated limits and revocation for allowed protective calls.	Allowlisted protocols/functions/assets; per-call/day cap; expiry; nonce; max slippage; pause; revoke.
riskos-executor-v1	Executes a pre-authorized plan submitted by a keeper when policy conditions are met.	No arbitrary call; verified adapter; replay protection; keeper cannot redirect proceeds; post-conditions.
riskos-attestation-v1	Anchors a hash of a signed risk snapshot or eligibility calculation.	Versioned signer set; timestamp/block; revocation; no price authority unless explicitly governed.
riskos-rewards-v1	Merkle/claim distribution only if contracted to distribute partner rewards.	Root governance, claim proofs, expiry, recovery, independent calculation/audit.

7.3 Why automation still needs offchain infrastructure

Clarity contracts do not wake themselves when a risk threshold changes. A keeper or user must submit a transaction. Therefore automation requires: monitoring, threshold evaluation, quote acquisition, signed/registered policy proof, transaction submission, fee funding/sponsorship strategy, idempotency, competing-keeper handling, confirmation tracking and emergency pause. A contract alone does not solve the automation product.

7.4 Contract engineering requirements

- Write a formal specification and threat model before implementation.
- Clarinet unit tests, invariant tests and integration tests against forked/replicated protocol interfaces.
- Use explicit traits for supported adapters; deny unknown principals and contract upgrades by default.
- No upgrade key held by one EOA. Use multisig + timelock + public change notice, or immutable versioned deployments.
- Independent Clarity audit, remediation review and public report before mainnet.
- Bug bounty with scoped contracts and payout budget.
- Emergency pause must stop new delegated execution but preserve revocation and safe user exits.
- Onchain event schema for policy creation, changes, execution, rejection, pause and revocation.

8. Module 4 - Distribution SDK and widget

8.1 Deliverables

Package	Audience	Contents
@riskos/client	Backend and frontend developers.	Typed REST client, retries, pagination, idempotency, webhook verification.
@riskos/react	Wallets and dApps.	Hooks for positions, risks, rewards, market health and transaction intents.
@riskos/widget	White-label product teams.	Embeddable portfolio/risk panel and Protect Position flow; theme tokens and localization.
RiskOS OpenAPI	Any integrator.	OpenAPI 3.1 contract, examples, changelog and generated clients. [S33]
Webhook kit	Institutions/protocols.	Signature verification, replay protection, event types and retry guidance.
Partner console	Integration teams.	API keys, usage, webhook replay, address scopes, health and sandbox.

8.2 Integration experiences

- Xverse/Leather: warning badge at asset/position level, portfolio risk summary and deep-linked protect transaction.
- Zest/Granite: independent user-configured alerts, scenario preview and cross-protocol exposure without implying an audit endorsement.
- Bitflow: LP range, fee, incentive and executable exit information in the liquidity position context.
- Stacking pools: net yield, operator/custody disclosures, capacity, maturity and redemption status.
- Fireblocks/Fordefi/Copper/BitGo-style systems: policy webhook, audit record, unsigned transaction package and approval state callback.
- Treasuries: wallet groups, maturity ladders, CSV/JSON exports, accounting cutoffs and approval history.

8.3 Widget security

- Iframe isolation for third-party embedding by default; strict Content Security Policy and allowed-parent origins.
- Host-provided theme is validated tokens, not arbitrary CSS/HTML.
- Transaction preview always displays the RiskOS origin and verified contract identity.
- No sensitive bearer token in URL, local storage or analytics events.
- Versioned bundle with Subresource Integrity where applicable; rapid kill switch for compromised releases.
- Accessibility target WCAG 2.2 AA, keyboard operation and non-color-only severity cues.

8.4 Commercial API controls

- API keys hashed at rest, prefix-identifiable, rotatable and scope-limited.
- OAuth 2.1/OIDC for enterprise user delegation; service accounts for server-to-server use.
- Per-tenant quotas, burst limits, address limits, export limits and cost controls.
- Signed webhooks using timestamped HMAC or asymmetric signatures; replay window and event ID.
- Usage metering by billable endpoint and successful data freshness class, not raw internal fan-out.
- Contractual SLA, support tier, data disclaimer, deprecation policy and incident communication.

9. API surface and behavior

9.1 Core endpoints

Endpoint	Purpose	Cache / freshness
GET /v1/address/{address}/positions	Normalized positions and provenance.	15-30s cache; block-linked ETag.
GET /v1/address/{address}/risk	Current findings, score components and scenarios.	Recompute on material event/price move.
GET /v1/address/{address}/rewards	Accrued, claimable and paid rewards with net APY.	Protocol-specific; expose as-of.
GET /v1/address/{address}/incentive-eligibility	Rule evidence and time-weighted qualification.	Event-updated plus daily signed snapshot.
POST /v1/risk/simulate	User-defined shocks and post-action simulation.	No shared cache for private tenant inputs.
POST /v1/actions/plan	Generate protective alternatives.	Short-lived intent.
POST /v1/actions/{id}/quote	Refresh route, fee and min-out.	Expires in seconds/minutes.
POST /v1/actions/{id}/wallet-request	Return compatible wallet transaction request.	One-time/idempotent.
GET /v1/transactions/{txid}	Track confirmation and reconciliation.	Event-updated.
GET /v1/markets/liquidity	Depth, slippage, volumes and source health.	10-30s by market.
GET /v1/system/health	Public dependency status and freshness.	Near real time.
POST /v1/webhooks	Create tenant-scoped subscription.	Authenticated.

9.2 Error model

```
{
  "type": "https://docs.riskos.xyz/errors/stale-source",
  "title": "Required price source is stale",
  "status": 409,
  "code": "PRICE_SOURCE_STALE",
  "detail": "DIA BTC/USD age exceeds the action safety window",
  "requestId": "req_...",
  "retryable": true,
  "sourceHealth": [{"source": "dia", "ageSeconds": 944, "limitSeconds": 180}]
}
```

9.3 API rules

- OpenAPI 3.1 is the normative contract; JSON Schema validates requests and responses.[\[S33\]](#)
- Amounts and prices are decimal strings, never JSON floating point.
- Cursor pagination only; stable ordering and maximum page sizes.
- All POST workflows accept Idempotency-Key and return replay-consistent results.
- Rate-limit headers, request IDs, source timestamps and model versions are standard response metadata.
- Breaking changes require a new major path or media-type version and at least 90 days' partner notice.
- Generated TypeScript clients and contract tests ship from the same OpenAPI artifact.

10. Data architecture

10.1 Core tables

Table	Key fields	Retention
chain_blocks	chain, height, hash, parent, canonical, observed_at.	Permanent canonical + reorg history.
chain_events	event_id, txid, block, contract, topic, raw payload, canonical.	Permanent; encrypted raw quarantine.
protocol_registry	protocol, network, version, contract, interface_hash, active range, approval.	Permanent version history.
assets	asset_id, principal, decimals, lineage, issuer, metadata version.	Permanent version history.
positions	position_id, address, protocol, type, current normalized state.	Current + tombstones.
position_snapshots	position_id, block/time, amounts, valuations, source set.	Hot 24 months; archive thereafter.
prices	asset, source, price, confidence, source_timestamp, observed_block.	Hot 24 months; archive.
risk_results	risk_id, snapshot, model/version, components, evidence, expiry.	At least 7 years for enterprise or policy-defined.
action_intents	intent, risk, calls, simulation, expiry, tenant, status.	At least 7 years for enterprise.
transaction_tracking	intent, txid, mempool/confirmation/reconcile state.	Permanent operational/audit record.
alerts	rule, occurrence, severity, delivery, acknowledgment.	Policy-defined; minimum 2 years.
reward_intervals	position, interval, eligible value, reward type and proof.	Program duration + 7 years.

10.2 Event identity and idempotency

A chain event key should combine network, canonical block hash, transaction ID, event index and contract principal. Every ingest operation is upsert-idempotent. Reorg processing marks prior events noncanonical, rolls back derived rows using lineage references and recalculates affected snapshots and alerts. Notification delivery uses a separate immutable occurrence ID so a reorg correction can be communicated explicitly.

10.3 Data lineage

Every public number must trace to inputs. Store raw-event IDs, read-only call request/response hash, protocol registry version, price observations, formula/model version and calculation timestamp. The API should offer a compact provenance object and an enterprise evidence endpoint. This is the difference between a credible risk product and a visually polished tracker.

10.4 Backfills and replays

- Bootstrap from protocol deployment blocks, not genesis, using documented start blocks in the registry.
- Replay historical chainhooks or API blocks into an isolated staging namespace before promotion.[\[S9\]](#)
- Compare event-derived state to current contract reads at defined checkpoints.
- Version transformation code so historical backfills do not mutate already-issued reports silently.
- Use golden addresses with known positions across each contract generation as regression fixtures.

11. Pricing, APY and incentive calculations

11.1 Price hierarchy

Use case	Primary value	Secondary checks
Protocol liquidation risk	Exact price/oracle behavior used by that deployed protocol.	DIA and executable market price for disagreement.
Portfolio valuation	Robust median of healthy references with quality weights.	Protocol oracle and DEX TWAP.
Executable exit	Live route quote for exact size.	Mid/reference price to calculate slippage.
Stablecoin risk	Market price plus documented backing/redemption state.	Issuer/xReserve health and venue depth.
Historical performance	Point-in-time stored observations.	Never substitute today's price into old reports.

11.2 Effective net APY

Report yield as components, not one number: protocol interest, fees, staking rewards, token emissions/incentives, borrowing cost, transaction fees, pool/operator fee, realized IL and unrealized mark-to-market. Use money-weighted return for user performance and time-weighted return for strategy comparability. Mark projected annualization separately from realized return.

```

net_return = interest + trading_fees + staking_rewards + incentives
- borrow_interest - protocol_fees - operator_fees
- transaction_costs + realized_price_PnL + realized_IL_effect

realized_annualized_return = (ending_value / beginning_value)^(365 / elapsed_days) - 1
// Only show when the measurement window and cash-flow treatment are explicit.
```

11.3 90-day incentive implementation

The announced program starts at Bitcoin block 966,350, allocates 0.5 BTC/month to qualifying Zest USDCx debt positions backed by sBTC/STX with time-weighted LTV at least 20%, and 0.5 BTC/month to active in-range Bitflow liquidity in sBTC/USDCx and STX/USDCx. Final eligibility details and exclusions are partner-defined and must be ingested as a versioned ruleset, not inferred from the announcement.[\[S4\]](#)

- Persist every position-changing event and the interval between changes.
- Calculate LTV using the program's specified price source, snapshot cadence, rounding and excluded position types.
- For LPs, record active-range intervals and any required activity/deployment conditions.
- Produce a daily eligibility evidence record and compare it against partner-published results.
- Label estimated rewards until the protocol/partner confirms distribution.
- When rules change, close the old version interval and open a new one; never rewrite prior eligibility.

12. Authentication, teams and institutional workflow

12.1 Retail identity

- A public address can be viewed without login.
- To save alerts or request actions, issue a short-lived domain-bound challenge and verify a wallet signature using supported Stacks wallet methods.
- Session tokens are short-lived, httpOnly, SameSite and rotated; wallet address is not treated as a secret.
- High-risk preferences and notification changes require recent wallet re-authentication.
- Never request a seed phrase or raw private key; include an anti-phishing statement in every signing flow.

12.2 Institutional tenancy

Role	Permissions
Owner	Billing, security configuration, role administration, integrations.
Risk admin	Risk policies, thresholds, watchlists and incident acknowledgment.
Approver	Approve/reject prepared actions within policy; cannot alter the plan.
Operator	Create plans and exports; cannot approve above assigned limit.
Viewer/Auditor	Read-only positions, evidence, actions and audit history.
Service account	API-only scopes, address restrictions and expiry.

12.3 Approval workflow

Plan creation, risk review, quote refresh, approval and custody signature are separate states. Amount thresholds can require M-of-N application approvals before a payload is handed to the custodian. RiskOS records approvals but does not replace a custodian's transaction policy engine. Any state or price drift beyond limits invalidates prior approvals.

13. Alerts and notification engine

13.1 Initial rule catalog

Rule	Default severity	Dedupe / reset
Health factor below 1.20 / 1.10 / 1.03	medium/high/critical	Re-alert on band change; resolve above 1.25 for 15 min.
Liquidation within 10% price move	high	Recompute on 1% collateral price move.
LP out of range	medium	One occurrence per excursion; reset on re-entry.
Exit slippage above 1% or tenant limit	medium/high	Re-alert if deterioration >25 bps.
Oracle stale/disagreement	high	Source-health incident; aggregate affected positions.
sBTC operation exceeds rolling p95	medium	Escalate to high at p99/absolute limit.
Incentive eligibility at risk	medium	Forecast breach before program cutoff.
Maturity within 30/7/1 days	info/medium/high	One alert per milestone.
Protocol contract changed/unsupported	critical	Persistent until approved adapter version.
Stablecoin deviation	high/critical	Hysteresis around recovery band.

13.2 Delivery

- In-app, email and webhook are mandatory. Push notification follows mobile/wallet partnerships; SMS is opt-in and high-cost.
- Critical alerts bypass digest but respect verified contact channels and rate-abuse limits.
- Every alert links to evidence, source timestamps, scenario and action alternatives.
- Webhook retries use exponential backoff, signed payloads and a dead-letter queue; partner console supports replay.
- Alert fatigue is measured through acknowledgment, mute and action conversion rates.

14. Security architecture and threat model

14.1 Crown jewels

- Integrity of protocol registry, adapter code, model versions and transaction intents.
- Tenant API keys, OAuth secrets, notification channels and institutional approval records.
- Availability and freshness of ingestion, pricing, risk and alerts.
- Reputation: a wrong liquidation price or malicious transaction payload can cause direct user loss even without custody.
- Optional future automation policy keys/roles and keeper infrastructure.

14.2 Threat matrix

Threat	Impact	Controls
Malicious/compromised upstream API	Wrong positions, prices or route.	Multiple sources, onchain reconciliation, provenance, circuit breaker.
Contract upgrade/address substitution	Funds sent to malicious contract.	Signed registry, explorer/source verification, interface hash, four-eyes approval.
Transaction tampering	User signs different call/amount.	Typed intent hash, same-object rendering/building, post-conditions, wallet review.
Stale state race	Protection no longer safe at signing.	Expiry, block drift limit, re-quote/re-simulate.
API key theft	Data exfiltration or quota abuse.	Scopes, address allowlists, rotation, anomaly detection, hashed storage.
BOLA / tenant escape	Private labels and policies exposed.	Object-level authorization and tenancy tests per OWASP. [S30]
Webhook forgery/replay	False critical signal or workflow action.	Signed timestamped events, nonce/event ID and replay window.
Reorg/double-processing	Duplicate positions/actions and false alerts.	Canonical block model, rollback, event idempotency.
Supply-chain compromise	SDK/widget or backend altered.	Pinned dependencies, SBOM, signed builds, provenance, secret scanning.
Admin compromise	Pause/registry/model abused.	SSO+MFA, least privilege, hardware-backed credentials, approvals and audit.

14.3 Secure development lifecycle

Adopt the NIST SSDF practices as the engineering baseline and OWASP API Security Top 10 as the API test baseline.[\[S30\]](#) [\[S31\]](#)

- Threat model and security requirements for every adapter/action before implementation.
- Protected branches, mandatory review, signed commits/releases, CI isolation and dependency pinning.
- SAST, dependency scanning, secret scanning, container scanning, IaC scanning and SBOM on every release.
- DAST and API authorization tests in staging; fuzz parsers and Clarity value decoders.
- Independent penetration test before public transaction building.
- Responsible disclosure policy and bounty for RiskOS code; coordinate disclosures involving partner protocols.
- Security review is repeated after new execution actions, contract adapters or automation.

14.4 Secrets and infrastructure

- Managed KMS/HSM for service signing keys and webhook secrets; no secrets in repository or client bundle.
- Short-lived workload identity instead of static cloud keys.
- Private subnets for data stores, least-privilege security groups and encrypted transport/storage.
- Separate production accounts/projects and break-glass process with post-use review.

- Immutable audit log export to a separately controlled account/bucket.
- Backups encrypted with restore tests; key rotation and offboarding runbooks.

15. Reliability, SLOs and operations

15.1 Production service objectives

Capability	Target SLO	Measure
API availability	99.9% monthly for read APIs; higher contractual tier later.	Valid requests served excluding agreed maintenance.
Position freshness	95% within 30s of observed finalized Stacks event; all within 2 min under normal operations.	Event observed to snapshot published.
Critical alert latency	95% within 60s after qualifying fresh state.	Risk occurrence to provider acceptance.
Cached API latency	p95 < 500ms; uncached portfolio p95 < 2s.	Server timing by endpoint/tenant.
Transaction intent integrity	100% registry and simulation validation.	No bypass path; build failure is safer than unchecked intent.
Recovery point	<5 min for application state; chain data replayable.	Backup/WAL and event replay.
Recovery time	<60 min for critical API; <4h for reports.	Quarterly recovery exercise.

15.2 Dependency health

Track each Stacks API/node, Chainhook stream, Bitcoin source, Emily endpoint, oracle, protocol API, quote route and notification provider independently. Expose green/amber/red state, last success, lag, disagreement and affected products. A healthy HTTP response with stale blockchain data is not healthy.

15.3 Incident levels

Level	Example	Response
SEV-1	Malicious/wrong transaction payload, material cross-tenant leak.	Immediate execution pause, page all responders, public notice, preserve evidence.
SEV-2	Critical risk/price wrong or alerts unavailable for broad cohort.	Pause affected adapter/action, page on-call, notify partners.
SEV-3	Single upstream degraded; safe fallback working.	Investigate in hours, status update if prolonged.
SEV-4	Minor UI/report defect without risk impact.	Normal backlog/SLA.

15.4 Required runbooks

- Stacks API/Chainhooks lag or reorg; Bitcoin provider disagreement.
- DIA/Pyth Lazer/oracle stale or divergent.
- Emily/sBTC deposit or withdrawal backlog.
- Protocol contract upgrade or unrecognized event schema.
- Incorrect risk calculation/model rollback.
- Transaction builder or SDK compromise.
- Database failover, cache loss, queue backlog and object-store recovery.
- API key/tenant compromise, webhook secret rotation and privacy incident.

16. Testing and verification strategy

16.1 Test pyramid

Layer	Required tests	Release gate
Pure math	Integer rounding, health, liquidation price, APY, time-weighted values, LP math.	Property tests + golden vectors; 100% critical branch coverage.
Adapter contract	Fixtures for every event/read-only response and contract version.	Schema contract tests; unknown fields tolerated safely, unknown semantics rejected.
Historical replay	Known liquidations, bridge operations, LP changes and staking cycles.	Position/risk result matches independently calculated truth.
Live integration	Testnet and read-only mainnet shadow comparisons.	Daily reconciliation within defined tolerance.
Transaction	Construct, decode, simulate, inspect post-conditions and reject tampering.	No unverified contract/function; before/after state passes.
API/security	AuthZ matrices, quotas, malformed Clarity values, fuzzing, OWASP cases.	No high/critical findings.
Resilience	Provider outage, stale data, reorg, duplicate webhook, queue poison and DB failover.	System degrades visibly and fails closed.
UI/accessibility	Critical journeys, keyboard, screen reader, mobile/responsive and signing clarity.	WCAG 2.2 AA for core flows.

16.2 Independent validation

- Validate at least 100 historical positions/events across supported protocols before public launch.
- Have Zest, Granite, Bitflow and Stacking DAO review their adapter's interpretation and test vectors without giving them control over RiskOS ratings.
- Use a separate implementation or spreadsheet to cross-check core formulas.
- Run a 2-4 week shadow period where actions are generated but cannot be broadcast.
- Publicly document known limitations, unsupported markets and model confidence.

17. Privacy, compliance and legal workstream

17.1 Product posture

A non-custodial analytics and transaction-preparation product materially reduces custody risk, but it does not automatically remove financial-services, consumer-protection, sanctions, privacy or marketing obligations. Obtain counsel in launch jurisdictions before enabling paid advice-like features, automated execution, incentive distribution or custody-platform workflows.

17.2 Required controls

- Terms of service, privacy notice, cookie/analytics controls, risk disclosures and protocol-specific disclosures.
- State clearly that scores and simulations are estimates and not guarantees; avoid personalized investment claims.
- Treat wallet addresses, labels, contact channels and linked identities as sensitive/potentially personal data; minimize collection.
- Provide deletion/export workflows for offchain account data while preserving legally required audit records.
- Publish subprocessor list, retention schedule, data-processing terms and security contact for B2B customers.
- Respect protocol and jurisdiction restrictions in execution UI; do not use RiskOS to bypass partner controls.
- Sanctions/address screening should be an explicit institutional policy integration with documented false-positive and appeal handling, not a hidden public-address judgment.
- Review software licenses and trademark/brand permissions for each protocol SDK, ABI and logo used in the widget.

17.3 USDCx integration caveat

Circle documents xReserve as infrastructure for USDC-backed tokens. Its current supported-chain page must be reconciled with the exact Stacks mainnet token and protocol usage before RiskOS publishes backing or redemption claims. Store the verified token principal, issuer, reserve domain and withdrawal path in the registry, and monitor changes.[\[S26\]](#) [\[S27\]](#)

18. Admin, governance and change management

18.1 Control-plane actions

Action	Required approval	Audit requirement
Add/update protocol contract	Two authorized reviewers, one technical and one security.	Source links, interface hash, explorer evidence, test results.
Enable transaction action	Security sign-off plus protocol integration sign-off.	Threat model, simulation vectors, allowlist diff.
Change risk formula/weight	Risk owner + independent reviewer.	Version, reason, backtest, affected results.
Override source health	Two-person time-limited break-glass.	Reason, scope, expiry, after-action review.
Pause adapter/action	Single on-call for emergency; review within 24h.	Incident/event link and restoration criteria.
Rotate signing/webhook keys	Security operator under documented runbook.	Key IDs, overlap window and completion evidence.

18.2 Model governance

- Models are semantic-versioned. Formula changes are major/minor; threshold tuning is a documented configuration version.
- Backtests include false-negative and false-positive rates, not only accuracy averages.
- Risk results retain the model version used; historical reports are reproducible.
- Machine learning is not required for MVP. Deterministic rules and transparent statistical models are safer and easier to validate.
- If ML is introduced, keep it advisory, monitor drift, preserve explanations and never let it directly generate unrestricted calls.

19. CI/CD and release engineering

- Monorepo packages: apps/web, apps/api, workers/indexer, workers/risk, packages/domain, packages/adapters/*, packages/sdk, packages/widget, contracts/optional, infra/.
- Trunk-based development with short branches, protected main, required tests and environment promotion.
- Reproducible container builds, SBOM, image signatures and provenance attestations.
- Database migrations are backward-compatible, reviewed and rehearsed on production-size staging data.
- Canary API deployment, shadow adapter execution and feature flags by protocol/action/tenant.
- SDK release has semver, changelog, migration guide, bundle scan and rollback CDN artifact.
- Infrastructure-as-code plan review and drift detection; no console-only production resources.
- Release checklist confirms provider quotas, contract registry, source health, dashboards, runbooks and support coverage.

19.1 Repository quality gates

Gate	Minimum
Lint/typecheck	Zero errors; strict TypeScript; no unchecked any in domain/adapters.
Unit/property tests	Critical financial math and decoders pass deterministic vectors.
Contract tests	OpenAPI and protocol adapter schemas compatible.
Security	No unresolved critical/high SAST/dependency/container findings.
Coverage	Critical domain 95%+ line and branch targets; overall target set by package risk.
Performance	API and replay benchmarks within regression budget.
Artifact	Signed image/package, SBOM and immutable version.

20. Delivery plan

20.1 Grantable MVP - 16 to 20 weeks

Phase	Weeks	Deliverables	Exit criteria
0. Partner lock	1-2	Zest, Bitflow, sBTC manifest; rule definitions; golden addresses; legal posture.	Written technical contacts and verified integration inputs.
1. Data foundation	3-5	Stacks/Bitcoin ingestion, registry, raw events, snapshots, reconciliation, observability.	Reorg/replay tests; source health dashboard.
2. Position API	6-8	sBTC, Zest v2 and Bitflow adapters; positions/rewards/eligibility endpoints.	100-address comparison; provenance exposed.
3. Risk engine	9-11	Lending health, TW-LTV, LP range, exit slippage, oracle/bridge health, scenarios.	100 historical cases; independently reviewed formulas.
4. Protection	12-14	Repay, add collateral, swap, remove LP; intent, quote, simulation and wallet signing.	Shadow period and external security test.
5. Distribution	15-17	TS/React SDK, widget, webhooks, partner sandbox and docs.	Two protocol/wallet pilots.
6. Launch hardening	18-20	Pen test remediation, SLOs, runbooks, status page, privacy/terms, beta.	Launch-gate checklist signed.

20.2 Full product - following 6 to 12 months

- Granite production adapter after formal interface validation.
- Stacking DAO stBTC and pooled staking, followed by direct Bitcoin Bond reporting.
- Treasury workspaces, approvals, custody-platform handoffs and evidence exports.
- Python SDK and additional wallet embeds.
- Advanced liquidity stress, look-through graph and historical risk analytics.
- Optional audited delegated-execution contracts only after signed-action product-market fit.
- SOC 2 Type I readiness then Type II evidence collection for institutional sales.

20.3 Scope discipline

Do not put all protocols and autonomous execution into the first grant milestone. The credible wedge is sBTC + Zest v2 + Bitflow + incentive/risk monitoring + user-signed protection + embeddable API/widget. The architecture in this report supports the complete product without making the first delivery impossible.

21. Team and ownership

Role	Initial allocation	Owns
Technical/product lead	1.0 FTE	Architecture, partner decisions, roadmap, product acceptance.
Protocol/Clarity engineer	1.0 FTE	Adapters, contract reads/actions, simulations, optional contracts.
Backend/data engineer	1.0-2.0 FTE	Ingestion, data model, API, queues, reconciliation.
Risk/data engineer	1.0 FTE	Financial math, models, backtests, evidence and monitoring.
Frontend/SDK engineer	1.0 FTE	Dashboard, wallet flows, SDK/widget, accessibility.
DevOps/SRE	0.5 then 1.0 FTE	IaC, observability, SLOs, incident and recovery.
Security engineer/auditor	Fractional + external	Threat models, reviews, pen test, contract audit.
Design/product research	0.5 FTE	User journeys, risk comprehension, integration design.
Legal/compliance	External/fractional	Terms, privacy, jurisdiction and institutional contracts.

21.1 Operating responsibilities

- Every protocol has a named adapter owner and backup.
- Every risk model has a technical owner and independent reviewer.
- On-call rotation begins before alerts are promised to users.
- Partner success owns integration acceptance, deprecation communications and SLA reviews.
- Security has authority to pause transaction building without product approval during an incident.

22. Infrastructure and operating cost

The following are planning ranges, not vendor quotes. Actual cost depends on indexed history, address count, price-provider licensing, retention, alert volume, support tier and redundancy.

Stage	Monthly infrastructure	Included baseline
Development/private alpha	\$300-\$1,000	Managed Postgres, small workers/API, object storage, basic monitoring, test providers.
Public beta	\$1,500-\$5,000	Multi-service staging/prod, Multi-AZ DB, Redis, queues, WAF, logs/traces, backups, email/webhooks.
Production growth	\$5,000-\$15,000+	Redundant chain providers/nodes, larger time-series store, on-call tooling, higher retention and traffic.
Institutional scale	Custom	Dedicated tenancy/options, compliance evidence, premium providers, 24/7 support and DR.

22.1 Non-infrastructure budget items

- External penetration test; Clarity audit only if RiskOS contracts are introduced.
- Bug bounty reserve and incident-response retainer.
- Market/oracle/API commercial licenses and protocol partner support agreements.
- Legal/privacy review, terms, DPA and insurance.
- Design/research, documentation, partner engineering and customer support.
- SMS/push/email usage and audit/compliance tooling.

23. Partner dependency checklist

Partner	Must obtain before production	Fallback
Stacks/Hiro	API/Chainhooks quotas, status/support, node recommendations, finality/reorg guidance.	Own Stacks API/node and replay pipeline.
sBTC/Emily	Endpoint/schema, statuses, limits, signer-health interpretation, mainnet/testnet manifests.	Onchain+Bitcoin monitoring; degrade unsupported status detail.
Zest	Signed current contract manifest, functions/events, egroupp math, oracle payload handling, incentive rules.	Read-only adapter only; disable actions until confirmed.
Granite	Current deployments, isolated-pool/soft-liquidation math, actions, oracle, SDK compatibility.	Defer adapter; show wallet assets only.
Bitflow	SDK/API credentials, LP schema, route/quote SLA, pool manifest, incentive rules.	Contract-indexed LP basics; disable routing if no reliable quote.
Stacking DAO/pools	stBTC contract/backing/exchange rate, redemption/maturity, fee/capacity and operator status.	Display token balance as unsupported derivative.
DIA/Pyth	Current Stacks deployment/support, feed IDs, timestamps, incident channels.	Protocol oracle only with degraded cross-check confidence.
Wallet/custody	Supported request methods, deep links, policy handoff, callbacks and branding/security review.	Copyable verified intent/manual official protocol link.

24. Launch acceptance criteria

24.1 Functional

- At least sBTC, Zest v2 and Bitflow positions resolve correctly for supported mainnet addresses.
- All core endpoints return provenance, as-of block/time, confidence and explicit unsupported items.
- Health factor, liquidation distance, time-weighted LTV, LP range, exit slippage, bridge status and net rewards pass golden cases.
- Repay, add collateral, swap and LP removal produce verified, simulated, expiring user-signable intents.
- SDK/widget integrated by at least one external product and exercised in a sandbox.
- Alerts deliver and reconcile without duplicates under normal reorg/retry scenarios.

24.2 Safety and reliability

- No critical/high open penetration-test finding.
- Protocol registry and execution allowlists have dual control and rollback.
- Stale/disputed sources block action construction and visibly degrade read results.
- Recovery and reorg drills meet RTO/RPO; runbooks and on-call contacts are current.
- SLO dashboards and public dependency status exist before paid API promises.
- Privacy, terms, disclosures, retention and support processes are approved.

24.3 Market validation

- 20 active wallets and at least \$1m equivalent monitored during pilot.
- Two protocol partners validate adapter outputs.
- One wallet/pool/protocol tests the widget or API in its product.
- Users act on or explicitly dismiss measured risk findings, proving the output changes decisions.
- Partner interviews confirm willingness to pay for data, alerts, execution infrastructure or support.

25. Principal risks to the RiskOS business

Risk	Why it matters	Mitigation
Protocol churn	Fresh contracts and undocumented changes can invalidate calculations.	Signed registry, versioned adapters, partner notices, fail closed.
Small market / weak willingness to pay	Good infrastructure may precede demand.	MVP around live incentive and embedded distribution; measure monitored capital and partner conversion.
Wrong risk output	Can cause direct loss or false confidence.	Explainable formulas, provenance, validation, disclaimers, independent review and safe degradation.
Upstream dependency	APIs/oracles can be unavailable or change.	Own indexing, multiple sources, reconciliation and contractual access.
Transaction liability	A malformed protective transaction is high impact.	Strict allowlists, simulation, post-conditions, security audit and user signature.
Automation expands regulatory/security surface	Delegation resembles asset management and increases loss impact.	Defer until validated; bounded contracts and counsel review.
Distribution partner conflict	Protocols may dislike independent ratings.	Transparent methodology, factual evidence, right-to-correct data but independent conclusions.
Nested exposure double count	Can badly distort portfolio risk.	Asset lineage graph and unit-level economic ownership.

26. Open decisions before engineering kickoff

Decision	Recommended default	Owner / deadline
MVP protocols	sBTC, Zest v2, Bitflow; Granite read-only if manifest is ready.	Product + partners, week 1.
Primary user	Active borrower/LP first; treasury features behind same API.	Product, week 1.
Backend framework	TypeScript/Fastify or NestJS; PostgreSQL/Timescale; Redis; managed queues.	Tech lead, week 1.
Workflow engine	BullMQ/SQS for MVP; Temporal if long-running institutional flows are immediate.	Tech lead/SRE, week 2.
Price provider licensing	Protocol oracle + DIA + executable Bitflow; add commercial market feed if terms require.	Risk/legal, week 2.
Public address lookup	Yes, strict quota and no private labels.	Product/privacy, week 2.
Transaction broadcast	Wallet broadcasts in MVP; RiskOS tracks.	Security/product, fixed.
New contracts	None in MVP.	Fixed unless grant scope changes.
Hosting region	Select based on users/partners; multi-AZ and documented data transfers.	SRE/legal, week 3.
Brand/name clearance	Check RiskOS naming/trademark/domain before launch.	Legal/brand, before public identity.

27. Recommended repository and service map

```

riskos/
apps/
web/ # Dashboard, public market pages, signing UX
api/ # REST, auth, tenant and partner console APIs
docs/ # OpenAPI reference, guides, changelog
workers/
chain-indexer/ # Stacks/Bitcoin/Chainhooks ingestion and reorgs
reconciler/ # Direct reads and mismatch handling
risk-engine/ # Models, scenarios, rule evaluation
notifier/ # Email, webhook, push, delivery state
reporting/ # CSV/PDF/JSON exports
packages/
domain/ # Money, block, position, risk and intent types
registry/ # Signed assets/contracts/interfaces/model config
adapters/{stacks,sbtc,zest,granite,bitflow,stackingdao,pox5}/
pricing/ # Source adapters, quality, disagreement, TWAP
simulation/ # Protocol-aware post-action state
client/ react/ widget/
contracts/ # Empty for MVP; optional policy/executor later
infra/ # IaC, dashboards, alerts, runbooks
fixtures/ # Golden chain events, read calls and addresses
openapi/ # Normative API contract and webhook schemas
security/ # Threat models, dependency policy, disclosures

```

27.1 Domain rules

- No package imports a protocol's raw shape outside its adapter.
- Money/amount types prohibit float and require asset/decimal context.
- Risk models accept immutable snapshots and return deterministic results.
- Transaction builder accepts only typed, validated intents from approved adapters.
- UI never constructs contract calls by concatenating strings.
- Registry changes and model changes are deployable independently but auditable together.

28. Final implementation recommendation

Build RiskOS as a protocol-neutral data and safety layer, not as another portfolio dashboard. The defensible asset is the normalized, reproducible risk graph and the transaction-intent infrastructure that wallets, protocols and treasuries can consume.

Build now: own ingestion and reconciliation; sBTC + Zest v2 + Bitflow adapters; time-weighted incentive analytics; liquidation and LP exit risk; user alerts; signed protective actions; OpenAPI/TypeScript SDK/widget. Build next: Granite, stBTC/pools, Bitcoin Bond reporting, treasury approvals. Defer: autonomous execution and new Clarity contracts until users repeatedly act on recommendations and partners demand automation.

The product succeeds if a user can answer four questions with evidence: Where is my Bitcoin capital? What can go wrong? How close am I? What exact safe action can I sign now? The API succeeds if another product can answer those questions without rebuilding RiskOS.

Appendix A - Complete build checklist

A.1 Product and partnerships

- Signed product requirements, supported markets, personas and exclusions.
- Protocol technical contacts and change-notice channels.
- Verified mainnet/testnet contract registry and deployment blocks.
- API/SDK terms, quotas, commercial rights, brand permissions and incident contacts.
- Golden test addresses/positions and partner output review.
- Pricing, support and pilot agreements for initial distributors.

A.2 Engineering

- Stacks/Bitcoin ingestion, reorg handling, backfills and reconciliation.
- Asset/contract/model registry with dual control.
- All adapters, pricing service, snapshot ledger and lineage graph.
- Risk models, scenarios, confidence, eligibility and alerts.
- Intent planning, quotes, simulation, wallet requests and transaction tracking.
- OpenAPI, SDKs, widget, webhooks, partner portal and admin console.
- Exports, audit evidence, multi-wallet tenancy and approval workflow.

A.3 Production readiness

- Threat models, security testing, penetration test and remediation.
- SLOs, dashboards, paging, runbooks, status page and incident process.
- Backups, restore drill, failover, capacity tests and cost controls.
- Terms, privacy, disclosures, retention, deletion/export and DPA.
- Support process, escalation ownership, feedback and analytics.
- Launch gates, canary, rollback and post-launch review.

Appendix B - Source and contract verification rules

- Require two independent official confirmations for every mainnet principal: protocol-controlled repository/documentation plus explorer/deployer evidence.
- Record the source URL, retrieval date, reviewer, interface/source hash and activation block.
- Never accept a contract address from social media, search snippets or an unverified community list alone.
- Monitor registry/deployer/governance events and official repositories for changes.
- A newly observed contract is unsupported until reviewed, tested and activated; do not silently follow proxies/upgrades.
- When official sources conflict, disable action construction and request partner confirmation.

B.1 Currently verified anchors

Anchor	Current public evidence	Implementation note
sBTC token	DIA guide identifies mainnet token SM3VDXK3WZZSA84XXKAF15NNZX32CTSG82JFQ4.sbtc-token.[S24]	Still verify against official sBTC registry before shipping.
DIA oracle	SP1G48FZ4Y7JY8G2Z0N51QTCYGBQ6F4J3J77BQC0.dia-oracle.[S24]	Read value and timestamp; apply product staleness limits.
Zest v2 deployer	SP1A27KFY4XERQCCRCARCYD1CC5N7M6688BSYADJ7; current deployment list includes v0-8-market and stBTC vault/strategy.[S15]	Import exact principals from deployment file; partner confirm newest revisions.
Bitflow	Official docs expose SDK/contracts/audits navigation.[S20] [S21] [S36]	Contract/LP schema must be captured in partner manifest.
Granite	Verified GitHub organization and active Immunefi scope.[S17] [S18]	Obtain exact production manifest before adapter/action enablement.
Stacking DAO	Public docs/product and contract repository.[S22] [S23] [S37]	Confirm stBTC-specific deployment, exchange rate and redemption model.

Appendix C - Reference links

Sources were accessed on 3 September 2026. Links are clickable. Product and protocol documentation changes quickly; the implementation registry must capture a reviewed snapshot rather than relying on this report indefinitely.

- S1 - Stacks 2026 Roadmap
<https://www.stacks.co/roadmap>
- S2 - Stacks Bitcoin Staking FAQ
<https://www.stacks.co/blog/bitcoin-staking-frequently-asked-questions>
- S3 - Pooled Bitcoin Staking on Stacks
<https://www.stacks.co/blog/pooled-bitcoin-staking-on-stacks-how-it-works>
- S4 - Stacks 90-Day DeFi Incentive Program
<https://www.stacks.co/blog/stacks-90-day-defi-incentive-program-announced-with-zest-and-bitflow>
- S5 - Stacks sBTC documentation
<https://docs.stacks.co/learn/sbtc>
- S6 - sBTC Bridge API reference
<https://docs.stacks.co/reference/api/sbtc-bridge>
- S7 - Emily API documentation
<https://docs.stacks.co/learn/sbtc/emily-api>
- S8 - sBTC open-source repository
<https://github.com/stacks-sbtc/sbtc>
- S9 - Hiro Chainhooks
<https://docs.hiro.so/en/tools/chainhooks>
- S10 - Stacks API reference
<https://docs.stacks.co/reference/api>
- S11 - Stacks Connect - connect wallet
<https://docs.stacks.co/stacks-connect/connect-wallet>
- S12 - Stacks Connect reference
<https://docs.stacks.co/reference/stacks.js/stacks-connect>
- S13 - Clarinet local blockchain development
<https://docs.stacks.co/clarinet/local-blockchain-development>
- S14 - Zest v2 contracts and architecture
<https://github.com/Zest-Protocol/zest-v2-contracts>
- S15 - Zest v2 mainnet deployment
<https://github.com/Zest-Protocol/zest-v2-contracts/blob/main/mainnet/README.md>
- S16 - Zest v2 market architecture
<https://github.com/Zest-Protocol/zest-v2-contracts/blob/main/docs/High-Level-Overview.md>
- S17 - Granite Protocol GitHub
<https://github.com/GraniteProtocol>
- S18 - Granite Immunefi program
<https://immunefi.com/bug-bounty/granite-protocol/information/>
- S19 - Bitflow developer overview
<https://docs.bitflow.finance/bitflow-documentation/developers/overview>
- S20 - Bitflow SDK
<https://docs.bitflow.finance/bitflow-documentation/developers/bitflow-sdk>
- S21 - Bitflow contracts and documentation
<https://docs.bitflow.finance/>
- S22 - Stacking DAO documentation
<https://docs.stackingdao.com/stackingdao>
- S23 - Stacking DAO smart contracts
<https://github.com/StackingDAO/contracts>
- S24 - DIA Stacks oracle guide
<https://www.diadata.org/docs/guides/chain-specific-guide/stacks>
- S25 - Pyth on Stacks guide
<https://docs.pyth.network/price-feeds/core/use-real-time-data/pull-integration/stacks>
- S26 - Circle xReserve overview
<https://developers.circle.com/xreserve>
- S27 - Circle xReserve supported chains
<https://developers.circle.com/xreserve/references/supported-blockchains-and-domains>
- S28 - Xverse developer documentation
<https://docs.xverse.app/>

S29 - Stacks on Fireblocks walkthrough

<https://www.stacks.co/blog/how-to-access-bitcoin-yield-in-fireblocks-with-stacks-full-walkthrough>

S30 - OWASP API Security Top 10

<https://owasp.org/www-project-api-security/>

S31 - NIST Secure Software Development Framework

<https://csrc.nist.gov/pubs/sp/800/218/final>

S32 - OpenTelemetry documentation

<https://opentelemetry.io/docs/>

S33 - OpenAPI Specification 3.1.1

<https://spec.openapis.org/oas/v3.1.1.html>

S34 - Stacks sBTC overview

<https://www.stacks.co/sbtc>

S35 - Zest v2 bug bounty

<https://immunefi.com/bug-bounty/zest-protocol-v2/information/>

S36 - Bitflow audits

<https://docs.bitflow.finance/bitflow-documentation/resources/audits>

S37 - Stacking DAO product overview

<https://www.stackingdao.com/>

Appendix D - Definition of done by module

Module	Definition of done
Unified Position API	Supported positions are complete, reconciled and provenance-aware; unknown/unsupported exposures are explicit; API is versioned, measured and documented.
Risk Engine	Protocol-exact critical math passes historical/golden tests; cross-protocol risks and degraded sources are explainable; every result is reproducible.
Protective Execution	All actions are allowlisted, simulated, expiring, human-readable and user-signed; confirmations reconcile; failures never masquerade as success.
SDK / Widget	A partner can integrate positions, risk, alerts and protect flows with documented auth, theming, accessibility, webhook security and backward compatibility.